Flaky Tests in UI: Understanding Causes and Applying Correction Strategies

Érica Sousa*
Federal University of Ceará (UFC)
Quixadá, Ceará, Brazil
ericams175@gmail.com

Carla Bezerra
Federal University of Ceará (UFC)
Quixadá, Ceará, Brazil
carlailane@ufc.br

Ivan Machado
Federal University of Bahia (UFBA)
Salvador, Bahia, Brazil
ivan.machado@ufba.br

ABSTRACT

Flaky tests are tests with non-deterministic behavior that can produce random and inconsistent results. While researchers has investigated Flaky tests in traditional unit tests, less is known about their occurrence in user interface (UI) tests. This study proposes an empirical analysis of 24 open-source projects to identify the main causes and correction strategies for Flaky tests in UI tests. Our analysis identified 8 categories of causes and 7 categories of correction strategies, with Race Condition, Logic Issues, and Test Dependency being the main causes of flaky tests in UI tests. Addition of Wait, Correction of Logic, and Ignored Test were the most commonly applied correction categories for resolving Flaky tests in UI tests. Specifically, 89% of Flaky tests caused by a Race Condition were corrected by adding Wait, while 100% of Flaky tests caused by Test Logic and Dependency Issues were fixed by patching the test logic. Our results provide insights into the occurrence of flaky tests in UI tests and can help Test Analysts and testers develop UI test automation projects with higher quality. This study contributes to the literature on flaky tests by providing empirical evidence of their occurrence in UI tests and identifying their main causes and correction strategies.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**; • **Human-centered computing → User interface programming**; • **General and reference → Empirical studies**.

KEYWORDS

Flaky tests, UI tests, Open-source software.

ACM Reference Format:
Érica Sousa, Carla Bezerra, and Ivan Machado. 2023. Flaky Tests in UI: Understanding Causes and Applying Correction Strategies. In *XXXVII Brazilian Symposium on Software Engineering (SBES 2023), September 25–29, 2023, Campo Grande, Brazil.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3613372.3613406

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SBES 2023, September 25–29, 2023, Campo Grande, Brazil
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0787-2/23/09…$15.00
https://doi.org/10.1145/3613372.3613406

1 INTRODUCTION

Flaky Tests are tests with non-deterministic behavior, which make it impossible to determine if the test case will either pass or fail in each execution [18]. These test cases often yield random success or failure when executed multiple times on the same system version without any changes to their source code [3, 10, 12, 13, 18]. Each occurrence of a false fault, induced by a flaky test, introduces uncertainty and disrupts the ability to determine whether the fault is genuinely due to system defects or the presence of a flaky test [19]. As a result, extensive debugging becomes necessary, involving an investigation of both the test code and the production code. However, this debugging process can be time-consuming and demanding, requiring substantial effort from developers and testers alike [12].

Flaky tests pose a significant challenge during regression testing, as they undermine the assumption that a failure indicates a defect [11]. For developers, these tests erode confidence in the test suite and result in wasted time spent debugging the faults [4, 15]. Although the factors contributing to the occurrence of flaky tests are still being studied [4, 25], existing research attributes their occurrence primarily to implicit waits and test dependencies [19].

Flaky tests present an even greater risk when integrated into a Continuous Integration (CI) environment, as each false failure obstructs the generation of new builds. This requires manual intervention and further delays the delivery process [12]. Various companies have observed the detrimental impact of flaky tests in recent years. For example, it was found that approximately 4.6% of tests across five Microsoft projects are unstable [11]. Another study [6], which examined 276 million test runs from the Chrome project, revealed that flaky tests cause 99.58% of all failures.

In automated testing, it is crucial to distinguish between two types of automation used in the development process: Unit Tests and User Interface (UI) Tests. Unit tests are designed to test the individual units of the system, such as the methods that constitute its source code. Typically, these tests are maintained by the developers themselves [7]. On the other hand, UI tests involve scripts that utilize frameworks to simulate user interactions with the interface of the System Under Test (SUT). They provide a means to validate the results obtained from the system, replicating the perspective of an actual user [19].

Unlike Unit Testing, UI Testing involves a distinct execution environment and automation process. Firstly, UI Tests encompass numerous events, such as user input handling, interactions with operating system or browser APIs, and downloading/rendering of various interface resources like images and scripts. These events frequently operate asynchronously, leading to the non-deterministic ordering of user events and other tasks. Secondly, reproducing flaky

tests in UI Testing is more challenging due to the involvement of two layers—the user interface and the code interface. This two-layered nature adds complexity to identifying and resolving flaky tests in UI Testing [19].

Therefore, this study aims to identify the primary causes of flaky tests in UI Tests across 24 open-source projects and examine the prevailing correction strategies employed for addressing flaky tests. To gain insights into the occurrence of flaky tests in UI Testing, we conducted an exploratory study analyzing 123 commits extracted from various open-source projects. The analysis revealed that Race Condition, Logic Issues, and Test Dependency were the root causes behind approximately 91% of the examined flaky tests. Furthermore, we identified that the most frequently employed correction strategies for addressing flaky tests in the analyzed commits were the Addition of Wait, Correction of Logic, and Ignoring the Random Test. By uncovering these findings, we aim to assist test analysts and testers in developing higher-quality UI Test automation projects while also providing valuable insights for future studies in this domain.

2 BACKGROUND AND RELATED WORK

2.1 Software test automation based on user interface (UI testing)

Automated tests are integral to the software development process, comprising scripted routines built with frameworks to execute the SUT without manual intervention. These tests encompass various forms of automation, as depicted by the Test Pyramid concept introduced by Mike Cohn [2]. The Test Pyramid serves as a visual representation, categorizing tests into System Tests, Integration Tests, and Unit Tests, based on their granularity, implementation speed, and complexity [21]. This hierarchical structure provides insights into the different levels of testing and their corresponding roles within the development cycle.

At the top of this pyramid, we find the system tests, also known as interface tests, which evaluate the application from the perspective of end users [19]. These tests are performed once the application has been fully integrated and completed, aiming to validate its functional requirements [21]. By simulating real-world scenarios, system tests navigate the application, interact with various elements such as buttons, and fill out forms. Their primary objective is to ensure that the observed behavior aligns with the expected functionality [2]. Due to their comprehensive nature, system tests tend to be more complex to maintain and possess a higher degree of automation complexity. Consequently, they are typically fewer within the software test suite [21].

UI Tests are scripts designed to interact with the SUT interface, providing a comprehensive assessment from the perspective of end users. These scripted tests ensure that the user interface layer aligns with the specified requirements and that all functionalities operate as intended [14]. By precisely simulating user interactions, UI tests allow testers to evaluate the application's behavior from a user-centric perspective. It is worth noting, however, that UI tests are limited in their access and capabilities. They do not possess the ability to directly interact with the internal code, manipulate internal variables, query the persistence layer, or monitor network requests [5]. Their focus primarily lies in evaluating the visible aspects of the application's interface and validating its behavior based on the expected user interactions.

While UI Tests and unit tests share a common objective, notable differences exist between them, particularly regarding execution environment and automation processes [19]. UI tests involve simulating user interactions, making API calls, and managing the download and rendering of interface components. These activities often occur asynchronously and in a non-deterministic order [5]. Consequently, reproducing flaky tests within the context of UI testing is more challenging, as it requires comprehensive coverage of all possible use cases and accurate simulation of user events [19].

Due to their nature as end-to-end tests, UI Tests tend to be more resource-intensive, slower, and less numerous than unit tests. They encompass a broader scope and assess the system behavior as a whole, interacting with various components and validating their integration [24]. However, this broader scope also renders UI tests more fragile. Minor changes in interface components can need corresponding modifications in these tests to ensure their continued accuracy and effectiveness.

2.2 Flaky tests

Flaky tests exhibit intermittent failures caused by concurrency issues [20]. They demonstrate varying successful and failed results for test cases that remain unchanged and are executed on the same SUT version [15]. Factors external to the test itself, such as the previous execution of other tests or the execution platform, can contribute to the occurrence of Flaky tests [15].

One prevalent category of Flaky tests is known as Order Dependent Tests, where the execution order of the tests impacts their outcome [23]. Platform-dependent tests can also be considered *flaky*, as they may pass on one machine but fail on another due to differences in the execution environment [3].

Dealing with flaky tests remains challenging for developers due to the lack of consensus and concrete guidelines. Current practices mainly involve refactoring flaky tests as they are identified, but developers lack automated tools or standardized approaches to support this process [8, 9, 16, 17]. Some developers ignore flaky tests by removing them from the regression test suite through comments or JUnit tags. However, this approach may not be ideal, as ignoring flaky tests can mask genuine system flaws [18].

2.3 Related work

Studies related to identifying common causes for flaky tests have been identified in the literature.

Luo et al. [13] present the details of a study applied to 51 open-source projects extracted from the Apache Software SVN repository. The authors analyzed 201 commits that fixed flaky tests to identify and classify root causes, detection methods, and correction strategies for flaky tests in Unit Testing.

Lam et al. [11] conducted a study on the overall lifecycle of flaky tests in six large-scale Microsoft projects, revealing the negative impact that flaky tests have on these projects. By analyzing the applied fixes, the authors found that the most common category of flaky tests in the analyzed projects is the Async Wait category (tests that make asynchronous calls without waiting for the return of the call). Thus, the authors confirm that categorizations of flaky

tests proposed by previous studies applied to open-source projects also apply to the proprietary Microsoft projects studied.

Romano et al. [19] identify the main causes and correction techniques for flaky tests in UI Testing projects from GitHub open-source. Using a methodology similar to that used by Luo et al. [13], they collected and analyzed 235 commits related to the occurrence of flaky tests in UI Testing found in 62 projects in web and Android environments. In this study, the authors identify common root causes of instability in UI Testing, the strategies used to identify unstable behavior, and the correction strategies used to fix flaky tests in UI Testing.

Gruber and Fraser [8] conducted a survey with 335 software developers and testers from different domains to understand how developers perceive flaky tests, how prevalent they are, and how much they affect developers' daily routines, as well as what they would like academia to do about the issue. As a result of the survey, the authors concluded that developers see flaky tests as a common and serious problem and are more concerned about losing confidence in test results than the computational costs of re-running unstable tests. Finally, developers expressed their desire for plugins to identify irregular code sections directly in their IDEs and for more training and information about flaky tests.

This study focuses on flaky tests, addressing their main causes, identification strategies, and correction techniques. While similar to the works by Luo et al. [13] and Lam et al. [11], which study flaky tests in Unit Testing, this research specifically examines flaky tests in UI Testing, similar to the study by Romano et al. [19]. To broaden the scope, projects using popular UI test automation tools like Selenium, Cypress, and Robot are selected to cover real-world scenarios. Additionally, the study identifies different causes and common fix strategies from Romano et al.'s work [19], aiming to understand the causes and patterns of flaky tests comprehensively. The findings have the potential to significantly enhance the quality and stability of automation tools in open-source projects.

3 STUDY SETTINGS

This study aims to investigate the causes of flaky tests in interface-based automated tests in open-source projects and identify the main correction strategies for the identified flaky tests. To achieve this goal, the following research questions (RQs) have been defined:

RQ_1: *What are the most common causes of flaky tests in UI Tests?* This research question focuses on identifying the prevalent causes of flaky tests in UI Tests. The insights gained from this investigation are expected to be valuable for Test Analysts, as they can avoid including these causes in their test scenarios. By analyzing commits related to flaky tests in UI Tests, the study aims to uncover the root causes behind this behavior.

RQ_2: *What are the most commonly used correction strategies for dealing with flaky tests?* This research question focuses on understanding how developers and testers typically address the challenge of flaky tests in UI Testing. By identifying the correction strategies that are most frequently used, the aim is to provide insights into the practical approaches adopted by practitioners to mitigate the occurrence of flaky tests in their projects effectively.

RQ_3: *What are the most commonly used correction strategies for each identified cause of flaky tests?* This research question analyzes each commit to determine the root cause of the problems encountered. By mapping all the causes, it becomes possible to identify the relationships between the root causes, the specific fixes implemented, and the correction strategies that were most frequently utilized to address each problem category.

3.1 Study Settings and Procedures

3.1.1 *Definition of Project Selection Criteria.*

The selection of projects for this research was based on specific criteria to ensure the relevance and suitability of the data collected. The following criteria were established for project selection:

- **UI Test Presence**: The projects had to either have UI Tests or include modules specifically dedicated to UI Testing using popular frameworks such as Selenium WebDriver,[1] Robot,[2] or Cypress[3]. This criterion ensured that the selected projects contained relevant UI Test automation code for analysis.
- **Commit Threshold**: The projects had to have a minimum of 100 commits. This criterion was set to filter out smaller or less active projects, focusing on projects with a substantial development history. By selecting projects with a significant number of commits, we aimed to ensure sufficient data for analysis and capture a more comprehensive understanding of flaky tests in UI Testing.

3.1.2 *Data Collection Process.*

The data collection was conducted using the GHTorrent tool, which provides a queryable offline mirror of GitHub API data [1]. By utilizing GHTorrent, we were able to identify candidate projects that potentially contained UI tests. We accessed the GHTorrent data through Google BigQuery, which offers more than 15TB of GitHub data for analysis.[4] To retrieve projects with UI tests, we formulated a simple SQL query using standard database query language. The query filtered projects with more than 100 commits and selected those that had references to specific terms in their descriptions, as defined in Section 3.1.1.

The results of the query were exported as JSON files, which can be accessed in the githubDataHandler repository.[5] These files contained important information about the projects, including the project name, the project's API reference, the project description available on GitHub, and the number of commits in each project.

Additionally, we utilized the GitHub API v3, a REST API, to access and retrieve commits and other relevant information from the project repositories. This allowed us to gather comprehensive data from GitHub, which could be processed and analyzed using various programming languages and tools.

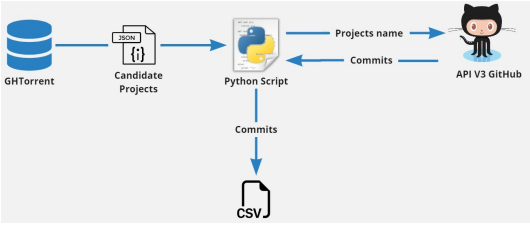
3.1.3 *Script Development for Commit Collection.*

To collect the commits used in this study, we developed a script that interacts with the GitHub API v3. The script takes JSON files from GHTorrent as input. These files contain project information, including project names. It then extracts the project names from the files and uses the GitHub API v3 to retrieve all the commits

[1] https://www.selenium.dev/documentation/webdriver/
[2] https://robotframework.org/
[3] https://www.cypress.io/
[4] https://ghtorrent.org/gcloud.html
[5] https://anonymous.4open.science/r/githubDataHandler-C1CA/resource/GHTorrent/

for each identified project. Next, it saves the selected commits in CSV files, accessible in the GitHub-DataHandler repository. Figure 1 illustrates the architecture of the script.

Figure 1: Script Architecture for Commit Collection



The CSV files contain the following columns:

- **repo_name**: The name of the project.
- **repo_url**: The link to the project.
- **url_commit**: The link to the commit.
- **message**: The commit message.

3.1.4 *Project Selection*.

In this step, we conducted the selection of projects for our study based on the criteria outlined in section 3.1.1. We utilized GHTorrent and the GitHub-DataHandler script to carry out the selection process. Initially, we queried the GHTorrent database using the search terms "Cypress", "Robot", "Selenium Webdriver", and popular front-end framework terms such as "react", "angular", "vue", "emberjs", "d3", "SVG", "web," and "bootstrap". Previous studies used these terms [13, 19]. We employed variations in writing the terms to ensure a broader selection of projects. For instance, to capture projects related to Robot, we used strings such as "Robot" and "robot" in the SQL query. Similarly, for Selenium, we utilized variations like "Selenium Webdriver", "selenium webdriver", "Selenium-Webdriver" and "selenium-webdriver".

In total, we selected 2,263 projects from GHTorrent, with a detailed breakdown provided in Table 1. These projects encompassed a range of technologies and frameworks, including nine projects related to Cypress, 1,915 projects related to Robot, 339 projects related to Selenium Webdriver, and 490 projects associated with various front-end frameworks. All the projects collected can be found in the GitHub-DataHandler repository for further reference and analysis.

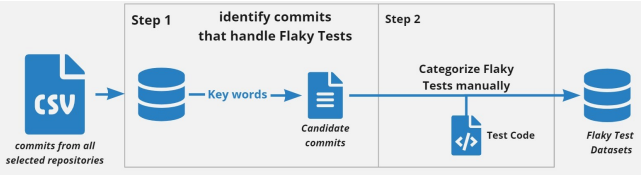
Table 1: Summary of Selected Projects from GHTorrent

research string	#projects
Cypress	9
Robot	1,915
Selenium Webdriver	339
Structures front-end	490

3.1.5 *Commit Extraction and Filtering*.

The first step in the data collection process was to extract the confirmation messages from all commits of the selected projects and store them in a CSV file. To identify commits that are likely related to flaky tests and their resolution, we employed a similar process used by Luo et al. [13] and Romano et al. [19]. Figure 2 illustrates the commit filtering process.

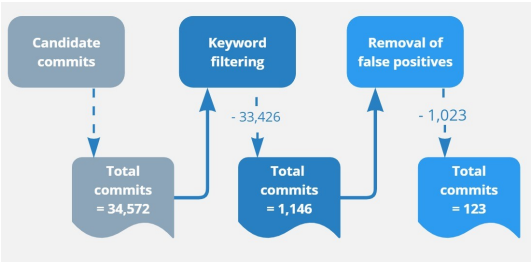
Figure 2: Commit Filtering Process



A total of 34,572 commits were collected from the 2,263 candidate projects. The initial filtering step involved searching for commits that likely address flaky tests (identification and/or fixing) by examining the commit confirmation messages using keywords proposed by Romano et al. [19]: *"flak"* and *"intermit"*. This step reduced the number of candidate commits to 1,146.

In the second step, a manual analysis of the confirmation messages was conducted to identify false positive commits. These are commits that contain at least one of the keywords but do not pertain to flaky tests in the UI test code. Through this analysis, a dataset consisting of 123 commits was obtained from 24 open-source projects for further study. Figure 3 provides a summary of the number of commits at each stage of the data filtering process.

Figure 3: Summary of Commits in the Filtering Process



The final dataset of 123 commits represents the relevant commits that will be analyzed to investigate the causes and correction strategies for flaky tests in UI tests.

3.1.6 *Data Analysis*.

In this step, we conducted a manual analysis of each commit selected in the previous filtering step to extract relevant information for addressing our research questions. For each commit, the following aspects were examined:

- **Commit messages:** The commit messages were inspected to gather details on how the *Flaky test* was fixed, the causes identified, and any relevant information for understanding the commit.
- **Developer comments:** The comments provided by developers in the commits were examined to gain further insights into how the *Flaky test* was initially identified, reproduced, and the implications of the potential fixes.

- **Changed code:** By analyzing the code changes introduced in each commit, we aimed to identify patterns and commonalities in the fixes for flaky tests. This analysis helped us categorize the fixes and generate a catalog of causes and correction strategies for flaky tests in UI tests.

Through our comprehensive analysis, we have obtained valuable insights to address our research questions. By examining the selected data, we successfully identified and mapped the frequent causes of flaky tests within the dataset, creating a comprehensive catalog of the correction strategies most commonly employed. These findings significantly contribute to our understanding of flaky tests and provide valuable guidance for enhancing the reliability and stability of UI tests.

Two experienced researchers with expertise in test automation and familiarity with the frameworks utilized in this study carried out the analysis process. In the initial stage, each researcher independently analyzed the commits, considering various aspects such as commit messages, developer comments, and code changes. Detailed observations and conclusions were meticulously recorded throughout this step in separate worksheets.

Subsequently, in the second stage, the researchers convened to compare and discuss the results of each analyzed commit. During this collaborative session, they shared their observations, delved into the nuances of each case, and actively sought points of convergence. This collective discussion fostered a more comprehensive analysis of the data. In the end, the comparisons and conclusions were consolidated into a final spreadsheet that meticulously and systematically recorded the analysis results for each commit.

3.1.7 ***Categorization of flaky tests.*** After manually inspecting the commits related to unstable UI tests, we identified 8 categories based on the categories proposed by Luo et al. [13] and Romano et al. [19], in which the root causes of instability in these tests can be classified. Each Cause category is explained below:

(1) **Race condition:** Occurs when there is an attempt to manipulate data or resources before they are fully loaded on the page. It is closely connected to the "External Interference" and "Concurrency Issues" categories identified in [13, 19]. External factors and the simultaneous execution of processes or threads can have an impact on data manipulation, resulting in intermittent failures during tests.
(2) **Logic issue:** Occurs when errors are present in the implemented logic of a test case, such as an incorrect sequence of steps or improper handling of test data [13, 19]. The works by Luo et al. [13] and Romano et al. [19] identify this category, emphasizing the significance of addressing logic-related errors in test cases to mitigate the occurrence of flaky tests.
(3) **Test dependency:** Occurs when test cases in a test suite are not isolated and their results are influenced by other test cases [13, 19]. It is closely related to the "External Interference" category identified by Luo et al. [13] and the "Data Dependency" category highlighted in Romano et al.'s work [19]. This category underscores the issue of interconnected test cases, where external factors or shared data can impact their results, leading to intermittent failures during tests.
(4) **Tool Failure:** It encompasses failures that occur within the testing framework or browser used for writing and executing tests, typically beyond the control of the test developers, as they are tied to the tools being utilized [13, 19]. Romano et al. [19] and Luo et al. [13] discuss this category, emphasizing the challenges posed by tool failures and their impact on test stability.
(5) **DOM Selector Problems:** It refers to issues encountered in UI tests related to the structures used to locate elements. For example, using an XPath that returns multiple elements instead of a single element can cause the test to interact with the incorrect element. The "DOM Selector Problems" category proposed by Romano et al. [19] addresses these issues. Both categories highlight challenges with the test's interaction with the page, including problems with element identification, element location, and the impact of animations on test execution. Such issues can lead to intermittent failures and present difficulties in test maintenance [19].
(6) **Randomness:** Occurs when a test case relies on randomly generated values and fails for specific generated values. This category is directly described in the work of Luo et al. [13], referring to issues with randomly generated values during test execution that can affect the test results.
(7) **External Interference:** Refers to any external interference in the SUT that is beyond the control of the test execution and leads to test failures. It encompasses scenarios where changes occur in test system data from unknown sources during test case execution, resulting in inconsistencies between the test system's actual data and the expected test case data. The category of "External Interference" is discussed in the works of Luo et al. [13] and Romano et al. [19], highlighting the impact of external factors on test results. These external interferences can arise from various sources, such as network changes, servers, or shared resources, and can unpredictably affect the test execution [13, 19].
(8) **Animation Timing Issues:** Occurs in tests that depend on the execution of an animation to perform the test or directly test the animation itself, as they are sensitive to timing differences in the execution environment. This category is described by Luo et al. [13], referring to issues with the execution of animations during the test execution that can affect the test result.

Additionally, we have identified seven categories of strategies for addressing flaky tests, based on the categories proposed by Luo et al. [13] and Romano et al. [19]. They are explained next:

(1) **Wait addition:** Refers to adding delays between actions that involve searching for or interacting with interface elements. It aligns with the "Delay between interactions" category proposed by Romano et al. [19] and the "Implicit Waits" category discussed by Luo et al. [13]. Both categories emphasize the use of wait strategies to account for page loading times or handle delays in the interface. While adding waits can enhance test stability, it is important to note that it may introduce variability in wait times, potentially leading to unstable tests.
(2) **Logic correction:** This category involves changing the test logic, which can introduce flakiness. It aligns with the "Test Logic" category proposed by Romano et al. [19], emphasizing

that changes in test logic can potentially cause flaky tests. Similarly, Luo et al. [13] categorize this as "Logic Issues," referring to problems in the test logic that can result in test failures. While modifications to test logic may be necessary to address issues, they can also introduce new problems and lead to flaky tests. Therefore, careful consideration and thorough review of changes in test logic are crucial to prevent the introduction of flakiness. However, successful corrections to the test logic can contribute to improving test stability.

(3) **Ignored test:** This category encompasses tests that are marked as random and automatically skipped during test execution. It aligns with the "Ignored Tests" category proposed by Romano et al. [19], which refers to tests that are skipped during execution. Additionally, this category can be justified by the "Random Tests" category mentioned by Luo et al. [13]. When a test is marked as random and skipped during execution, it does not contribute to test stability. Ignored or random tests can indicate potential flakiness issues since they may not adequately test the desired functionality and introduce instability in test results.

(4) **Increased wait time:** This category involves increasing the existing wait time for test operations. It align with the "Increased Time Limit" category proposed by Romano et al. [19], which suggests extending the time limit for test operations to address flakiness and improve test stability. This category is also supported by the "Increased Wait Time" category mentioned in Luo et al. [13], emphasizing the need to increase the wait time for interacting with UI elements. Increasing the wait time can accommodate variations in page loading time or ensure that animations are complete before proceeding with the next interaction. However, it is crucial to exercise caution, as increasing the wait time may introduce fluctuations that can lead to unstable tests [13].

(5) **Change in element access method:** This category involves making changes to the mechanism used to access UI elements. It aligns with the "Change in Element Access" category introduced by Romano et al. [19], which addresses modifications in the way UI elements are accessed to mitigate flakiness and enhance test stability. Additionally, it corresponds to Luo et al.'s "Element Location Issues" category [13], which focuses on challenges associated with accurately locating UI elements. By adjusting the approach to element access, it is possible to address flakiness issues and improve the reliability of UI tests.

(6) **Test removed from the test suite:** This category encompasses tests that are excluded or removed from the test suite. It aligns with the "Excluded Tests" category mentioned by Romano et al. [19] and the "Removed Tests" category. When a test is removed or excluded, it is no longer executed, potentially impacting the overall stability of the test suite.

(7) **Changing environment configuration:** This category involves making modifications to the execution environment settings. It aligns with changes in the test environment configuration, which can be implemented to address flakiness and enhance test stability. It corresponds to the "Test Environment Configuration" category identified by Romano et al. [19], which discusses adjustments in the test environment settings. Additionally, it is supported by the "Configuration Change" category outlined in Luo et al.'s work [13], which addresses changes in the execution environment settings that may impact test behavior.

A total of 123 commits were analyzed in this study, involving 24 projects, which provided a diverse set of samples of unstable UI tests for Web systems. The analyzed unstable UI tests were predominantly written in Java (61.8%), followed by TypeScript (13.8%), JavaScript (12.2%), Ruby (3.3%), and other languages (1.6%). The dataset can be accessed in the GitHub repository.[6] Table 2 presents detailed information about the projects used in our analysis.

Table 2: Source Projects for Data Extraction and Analysis

Projects	# flaky tests	# Commits	LOC
selenium	73	28,461	803,475
influxdb	11	35,142	384,589
acceptance-test-harness	10	5 307	33,016
selenide	4	3,908	64,004
cypress-example-kitchensink	3	924	42,193
metabase	2	23,148	687,463
platform	2	25,101	1 209,905
Selenium2	2	2,738	770,393
cypress-documentation	1	6,238	73,282
e2e-testsuite-platform	1	272	6,522
tor-browser-selenium	1	330	1 455
php-webdriver	1	1,065	12 263
server	1	529	16,199
ionic-framework	1	11,685	364,559
se34euca	1	545	21,912
next.js	1	11,690	462,341
angular	1	24,832	611,781
gatsby	1	20 358	859,260
react-styleguidist	1	2,162	179,734
react-cosmos	1	1 ,949	31,864
playground	1	516	12,322
opencrvs-farajaland	1	1,925	34,967
node.js	1	8,703	76,740
SwagPaymentPayPal	1	951	69,618

4 RESULTS AND DISCUSSIONS

4.1 RQ_1: What are the most common causes of flaky tests in UI Tests?

To answer RQ_1, we analyzed the collected flaky tests, the associated code changes, and the problems related to the commits to identify potential causes of test instability. Out of the 123 analyzed commits, 21 did not provide sufficient information to determine their cause. We observed that the majority of reported causes for flaky tests were race conditions (74) and logic issues (15). In the following sections, we discuss the main identified causes of flaky tests.

4.1.1 Add Delay. To reduce the chance of encountering unstable behavior caused by a Race Condition, 53% of the tests added a delay between actions involving searching or interacting with interface elements. These delays can be categorized as implicit and explicit waits. When using an implicit wait, the test is instructed to poll the DOM for a certain period of time when trying to find any non-immediately available element. An example is shown in Figure 5, where a Thread.sleep was added before capturing information for an assertion on line 73.

[6]https://github.com/EriccaSousa/githubDataHandler/blob/main/resource/Flakys_Planilha_Oficial.csv

Figure 4: Flaky tests correction strategy categories

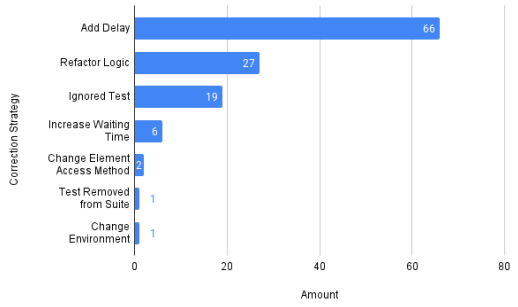


Figure 5: Example of adding a delay (1)

```
67 +   public void
       testShouldFocusOnTheReplacementWhenAFrameFollowsALinkToA_TopTargettedPage() throws
       Exception {
68         driver.get(framesetPage);
69
70         driver.findElement(By.linkText("top")).click();
71
72 +       // TODO(simon): Avoid going too fast when native events are there.
73 +       Thread.sleep(1000);
74 +
75         assertThat(driver.getTitle(), equalTo("XHTML Test Page"));
76         assertThat(driver.findElement(By.xpath("/html/head/title")).getText(),
77                    equalTo("XHTML Test Page"));
```

On the other hand, explicit waits allow you to specify a condition for the test to pause its execution until the condition is met. An example is shown in Figure 6, where a wait with the condition to only perform the click when the element with the ID "prompt" is present. This strategy ensures that the rest of the test script is not executed until the delay is over, giving the asynchronous call additional time to complete before proceeding.

Figure 6: Example of adding a delay (2)

```
317       public void testPromptShouldUseDefaultValueIfNoKeysSent() {
318         driver.get(promptPage("This is a default value"));
319
320 +       wait.until(presenceOfElementLocated(By.id("prompt"))).click();
321         Alert alert = wait.until(alertIsPresent());
322         alert.accept();
```

4.1.2 Refactor Logic. Some tests exhibited random behavior due to errors in the implementation logic. To address this issue, various changes were made to the logic of the tests in 21% of the analyzed commits to improve the intended purpose of the test and eliminate the flaky test. These changes include correcting the way elements are accessed, adjusting the order of steps in the interaction with the interface, generating data, among others.

An example of logic correction is shown in Figure 7, where the download() function is used to download the EXE file when the element with the text "Download EXE file" is located on the page. The downloaded file name must match any sequence of alphanumeric characters before the .exe extension. Then, two assertions are performed to verify that the downloaded file name starts with "tin.exe" and that the file size equals 43 bytes.

Figure 7: Example of refactor logic (1)

```
153       @Test
154       void downloadsPotentiallyHarmfulWindowsFiles() throws IOException {
155 +       File downloadedFile = $(byText("Download EXE
          file")).download(withNameMatching("\\w+\\.exe"));
156
157         assertThat(downloadedFile.getName()).startsWith("tiny.exe");
158         assertThat(Files.size(downloadedFile.toPath())).isEqualTo(43);
```

In Figure 8, there was a change in the regular expression passed to the withNameMatching() function. Previously, the regular expression was \\w+\\.exe, which matched any sequence of alphanumeric characters before the .exe extension. In the updated version, the regular expression has changed to tiny\\.exe\\., where the . was added before the last \\, which requires a literal dot after the filename and the .exe extension. This means the downloaded file must be named "tiny.exe." to be considered valid.

Figure 8: Example of refactor logic (2)

```
153       @Test
154       void downloadsPotentiallyHarmfulWindowsFiles() throws IOException {
155 -       File downloadedFile = $(byText("Download EXE
          file")).download(withNameMatching("tiny\\.exe.*"));
156
157         assertThat(downloadedFile.getName()).startsWith("tiny.exe");
158         assertThat(Files.size(downloadedFile.toPath())).isEqualTo(43);
```

4.1.3 Ignored Test. To deal with the occurrence of flaky tests, 15% of the analyzed commits removed them from their test suites, marking them as unstable tests. Removing the test from the test suite reduces code coverage.

Some tests are not completely removed from the test set. Instead, they are marked as skipped, which means that if the test fails, the entire test suite will not fail. An example of this correction strategy can be seen in Figure 9, where the "@CategorySmokeTest.class" tag in the right column is commented out, preventing the test case from being executed as part of the SmokeTest suite.

Figure 9: Example of an ignored test in the test suite

```
62        @Test
63 +      //TODO Re enable category once JENKINS-49524 is done
64 +      //@Category(SmokeTest.class)
65        public void login_and_logout() {
66
67            User user =
      realm.signup().fullname(FULL_NAME).email(EMAIL).password(PWD).signup(NAME);
```

RQ.2 Summary: The results were categorized into 3 main strategies: Add Delay, Logic Correction and Skipped Test. The most used strategy (53%) was to add a delay between actions that involve searching or interacting with interface elements, using implicit or explicit waits. Logic correction (21%) involved making various changes to the test logic to improve its intended purpose and remove the flaky test. Skipped tests (15%) involved removing them from their test suites, marking them

as unstable. Some tests were marked as ignored, meaning the entire test suite will not fail if the test fails.

4.2 RQ_3: What are the most commonly used correction strategies for each identified cause of flaky tests?

By analyzing the data, we have established relationships between the root causes of flaky tests and the adopted correction strategies. A summary of these relationships can be found in Table 3. To determine the relationship between the causes and fixes, we manually counted the number of times each fix strategy was used to resolve instances of flaky tests. This analysis enabled us to identify the most commonly employed correction strategies for each cause.

In Table 3, we observe the following results: flaky tests caused by External Interference, Randomness, Logic Issues, Screen Size, and Test Dependency were consistently resolved using Correct Logic operations in 100% of cases. This indicates the effectiveness of the logical fix approach for addressing these types of issues.

Regarding flaky tests caused by Race Condition, 89% of them were resolved by adding specific delays, while 8% were successfully mitigated by increasing the existing cooldown. This approach suggests that introducing controlled delays was an effective strategy for managing concurrency issues in testing. In the case of flaky tests resulting from DOM Selection Issues, we found that changing element access and applying logic fixes contributed equally, accounting for 50% of the corrections performed. This demonstrates the effectiveness of adjusting how DOM elements are selected and applying appropriate logic fixes when addressing these issues.

These findings underscore the importance of identifying the root causes of flaky tests and selecting the most suitable remediation strategies for each case. Understanding these relationships enables us to adopt more targeted and effective approaches to enhance the stability and reliability of automated tests.

RQ.3 Summary: Logic Correction was the most used strategy to solve flaky tests caused by External Interference, Randomness, Logic Issues, Screen Size, and Test Dependency. For flaky tests caused by Race Condition, the most used strategy was to Add Delay in 89% of the cases, while for flaky tests caused by DOM Selector Issues, the most used strategies were to change the element access and apply logic correction, each with 50% occurrence.

5 THREATS TO VALIDITY

Like any empirical study, this work also has threats to validity and limitations, which we present in this section. The classification of threats to validity described by Wohlin et al. [22] was used to discuss the threats in this work.

Construct validity threats: These threats relate to the extent to which the operationalization of variables and measurements align with the intended constructs and concepts. In this study, the classification of flaky tests and the identification of causes and correction strategies may be subject to interpretation or bias, potentially affecting the validity of the findings.

Internal validity threats: Internal validity concerns the degree to which the observed relationships between variables can be attributed to the causal effects being studied. In this study, potential threats to internal validity include subjective decisions made during data selection and analysis, which may introduce bias or errors in the identification of causes and correction strategies for flaky tests.

External validity threats: External validity refers to the generalizability of the findings to other contexts or populations. The selection of repositories for the study may introduce external validity threats if they do not represent a diverse range of projects or if the results are specific to certain frameworks, testing tools, or types of projects.

6 CONCLUSIONS

In this study, we conducted an investigation into the primary causes and correction strategies for flaky tests in automated UI tests across 24 open-source projects. Through the analysis of 123 commits associated with flaky tests, we aimed to provide valuable insights and practical guidance for resolving this recurring issue.

Our results revealed that Race Conditions (60%), Logic Problems (12%) and Test Dependency (3%) are the main causes of flaky tests in projects analyzed. When examining the correction strategies adopted in the commits of flaky tests, we identified that the most commonly used strategies to deal with these tests were Wait Addition (53%), Logical Correction (21%) and Test Skip (15%). Furthermore, we have identified specific remediation strategies for each identified cause, which can be beneficial in assisting developers and test analysts to approach flaky tests in a more effective and targeted manner. By providing a comprehensive understanding of flaky tests through this study, we aim to raise awareness of the challenges associated with inconsistent test results and offer potential solutions, ultimately assisting software development teams in enhancing the reliability and effectiveness of their automated testing efforts.

As a future direction, we propose replicating this study in industrial projects to compare and validate the obtained results. Additionally, we recommend conducting a thorough comparison of existing tools for identifying flaky tests, assessing their accuracy and usability across different stages of the development process. Furthermore, we suggest the development of a code review support tool that leverages the main categories of flaky test causes to identify code sections in commits that may result in flaky tests and suggest suitable correction strategies.

ARTIFACT AVAILABILITY

The material used and produced in our research is available at https://github.com/EriccaSousa/githubDataHandler

ACKNOWLEDGMENTS

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001; INES (www.ines.org.br), CNPq grant 465614/2014-0, FACEPE grants APQ-0399-1.03/17 and APQ/0388-1.03/14, CAPES grant 88887.136410/2017-00; and FAPESB grant PIE0002/2022.

Table 3: Relationship between root causes and correction strategies

Cause	#Occurrences	Correction Strategy	(%)
Race Condition	74	Add Delay	89%
		Increase in Wait Time	8%
		Removed Test	1%
		Change in Element Access	1%
Logic Issue	15	Logic Refactor	100%
Test Dependency	4	Logic Refactor	100%
Tool Failure	3	Ignored Test	100%
DOM Selector Issue	2	Change in Element Access	50%
		Logic Refactor	50%
Randomness	2	Logic Refactor	100%
External Interference	1	Logic Refactor	100%
Configuration Issue	1	Change in Environment Configuration	100%

REFERENCES

[1] [n.d.]. GHTorrent: GitHub's Data from a Firehose. In *MSR '12*, Jim Godfrey, Michael W. e Whitehead (Ed.). https://doi.org/10.1109/MSR.2012.6224294

[2] Mike Cohn. 2009. *Succeeding with agile* (1st ed.). Addison-Wesley Professional.

[3] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding flaky tests. In *27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 830–840. https://doi.org/10.1145/3338906.3338945

[4] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding Flaky Tests: The Developer's Perspective. In *27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2019)*. ACM, New York, NY, USA, 830–840. https://doi.org/10.1145/3338906.3338945

[5] Khaled El-Morabea and Hassaan El-Garem. 2021. Testing pyramid. In *Modularizing legacy projects using TDD*. Springer, 65–83. https://doi.org/10.1007/978-1-4842-7428-6_4

[6] Emad Fallahzadeh and Peter C. Rigby. 2022. The Impact of Flaky Tests on Historical Test Prioritization on Chrome. In *44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. ACM, New York, NY, USA, 273–282. https://doi.org/10.1145/3510457.3513038

[7] Gordon Fraser and Andrea Arcuri. 2014. A Large-Scale Evaluation of Automated Unit Test Generation Using EvoSuite. *ACM Trans. Softw. Eng. Methodol.* 24, 2, Article 8 (dec 2014), 42 pages. https://doi.org/10.1145/2685612

[8] Martin Gruber and Gordon Fraser. 2022. A survey on how test flakiness affects developers and what support they need to address it. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 82–92. https://doi.org/10.1109/ICST53961.2022.00020

[9] Sarra Habchi, Guillaume Haben, Mike Papadakis, Maxime Cordy, and Yves Le Traon. 2022. A Qualitative Study on the Sources, Impacts, and Mitigation Strategies of Flaky Tests. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 244–255. https://doi.org/10.1109/ICST53961.2022.00034

[10] Wing Lam, Patrice Godefroid, Suman Nath, Anirudh Santhiar, and Suresh Thummalapenta. 2019. Root causing flaky tests in a large-scale industrial setting. In *28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, New York, NY, USA, 101–111. https://doi.org/10.1145/3293882.3330570

[11] Wing Lam, Kıvanç Muşlu, Hitesh Sajnani, and Suresh Thummalapenta. 2020. A study on the lifecycle of flaky tests. In *ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. ACM, New York, NY, USA, 1471–1482. https://doi.org/10.1145/3377811.3381749

[12] Wing Lam, Stefan Winter, Anjiang Wei, Tao Xie, Darko Marinov, and Jonathan Bell. 2020. A large-scale longitudinal study of flaky tests. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 202 (nov 2020), 29 pages. https://doi.org/10.1145/3428270

[13] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In *22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014)*. ACM, New York, NY, USA, 643–653. https://doi.org/10.1145/2635868.2635920

[14] Abhishek Mishra. 2017. *Testing the User Interface*. Apress, Berkeley, CA, 407–432. https://doi.org/10.1007/978-1-4842-2689-6_13

[15] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A survey of flaky tests. *ACM Trans. Softw. Eng. Methodol.* 31, 1, Article 17 (oct 2021), 74 pages. https://doi.org/10.1145/3476105

[16] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2022. Surveying the Developer Experience of Flaky Tests. In *44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. ACM, New York, NY, USA, 253–262. https://doi.org/10.1145/3510457.3513037

[17] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2022. What Do Developer-Repaired Flaky Tests Tell Us about the Effectiveness of Automated Flaky Test Detection?. In *3rd ACM/IEEE International Conference on Automation of Software Test (AST)*. ACM, New York, NY, USA, 160–164. https://doi.org/10.1145/3524481.3527227

[18] Gustavo Pinto, Breno Miranda, Supun Dissanayake, Marcelo d'Amorim, Christoph Treude, and Antonia Bertolino. 2020. What is the vocabulary of flaky tests?. In *17th International Conference on Mining Software Repositories (MSR)*. ACM, New York, NY, USA, 492–502. https://doi.org/10.1145/3379597.3387482

[19] Alan Romano, Zihe Song, Sampath Grandhi, Wei Yang, and Weihang Wang. 2021. An empirical analysis of UI-based flaky tests. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 1585–1597. https://doi.org/10.1109/ICSE43902.2021.00141

[20] Arash Vahabzadeh, Amin Milani Fard, and Ali Mesbah. 2015. An empirical study of bugs in test code. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE Computer Society, USA, 101–110. https://doi.org/10.1109/ICSM.2015.7332456

[21] Marco Tulio Valente. 2020. *Modern software engineering*. https://engsoftmoderna.info/

[22] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in software engineering*. Springer Science & Business Media.

[23] Sai Zhang, Darioush Jalali, Jochen Wuttke, Kıvanç Muşlu, Wing Lam, Michael D. Ernst, and David Notkin. 2014. Empirically revisiting the rest independence assumption. In *International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 385–396. https://doi.org/10.1145/2610384.2610404

[24] Yixue Zhao, Justin Chen, Adriana Sejfia, Marcelo Schmitt Laser, Jie Zhang, Federica Sarro, Mark Harman, and Nenad Medvidovic. 2020. FrUITeR: A Framework for Evaluating UI Test Reuse. In *28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, New York, NY, USA, 1190–1201. https://doi.org/10.1145/3368089.3409708

[25] Wei Zheng, Guoliang Liu, Manqing Zhang, Xiang Chen, and Wenqiao Zhao. 2021. Research Progress of Flaky Tests. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 639–646. https://doi.org/10.1109/SANER50967.2021.00081